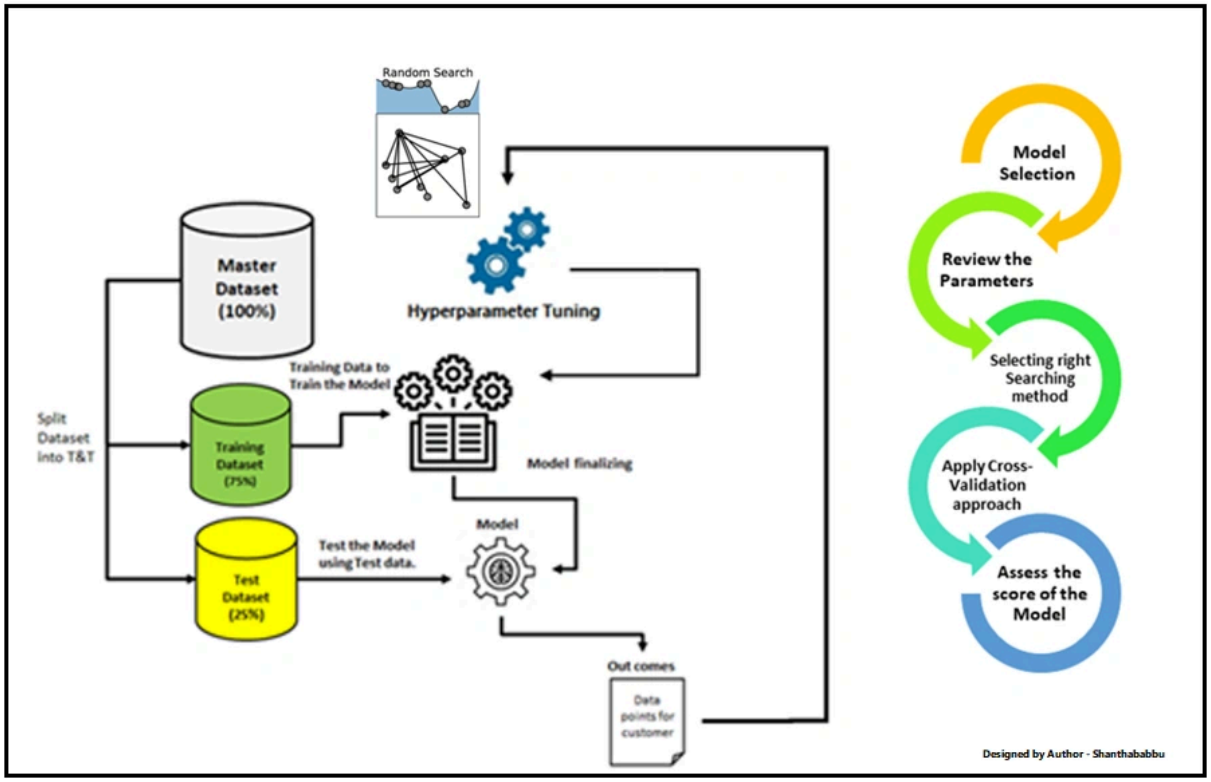
Hyperparameter Tuning in Deep Learning

Overview

In Deep Learning, hyperparameters are the configuration values you set before training a model.
They are not learned from data but control how the model learns.

- Parameters (weights, biases) → learned during training.
- Hyperparameters → fixed beforehand, chosen by the researcher.



Example:

- Learning rate = how fast the model learns.
- Batch size = how many samples the model sees before updating weights.

Without proper tuning, even the best neural network architecture may fail.

Why Hyperparameter Tuning Matters

- Neural networks are sensitive to hyperparameter settings.
- **A wrong learning rate can cause:**
    - Too high → divergence.
    - Too low → slow learning.
- **Choosing hyperparameters wisely can:**
    - Improve accuracy.
    - Prevent overfitting.
    - Reduce training time.

Key Insight: Often, a simple model with well-tuned hyperparameters can outperform a complex model with poorly chosen ones.

Common Hyperparameters to Tune

🔹 Learning Rate

- Controls step size in gradient descent.
- Most important hyperparameter.
- Tuned on log scale (0.1, 0.01, 0.001, 0.0001).

🔹 Batch Size

- Samples per weight update.
- Small batch → noisy but better generalization.

- Large batch → stable but may overfit.

🔹 Number of Epochs

- Full passes over training dataset.
- Too few → underfitting.
- Too many → overfitting.

🔹 Optimizer & Its Parameters

- Choice of algorithm (SGD, Adam, RMSProp).
- Momentum, beta values, learning rate decay.

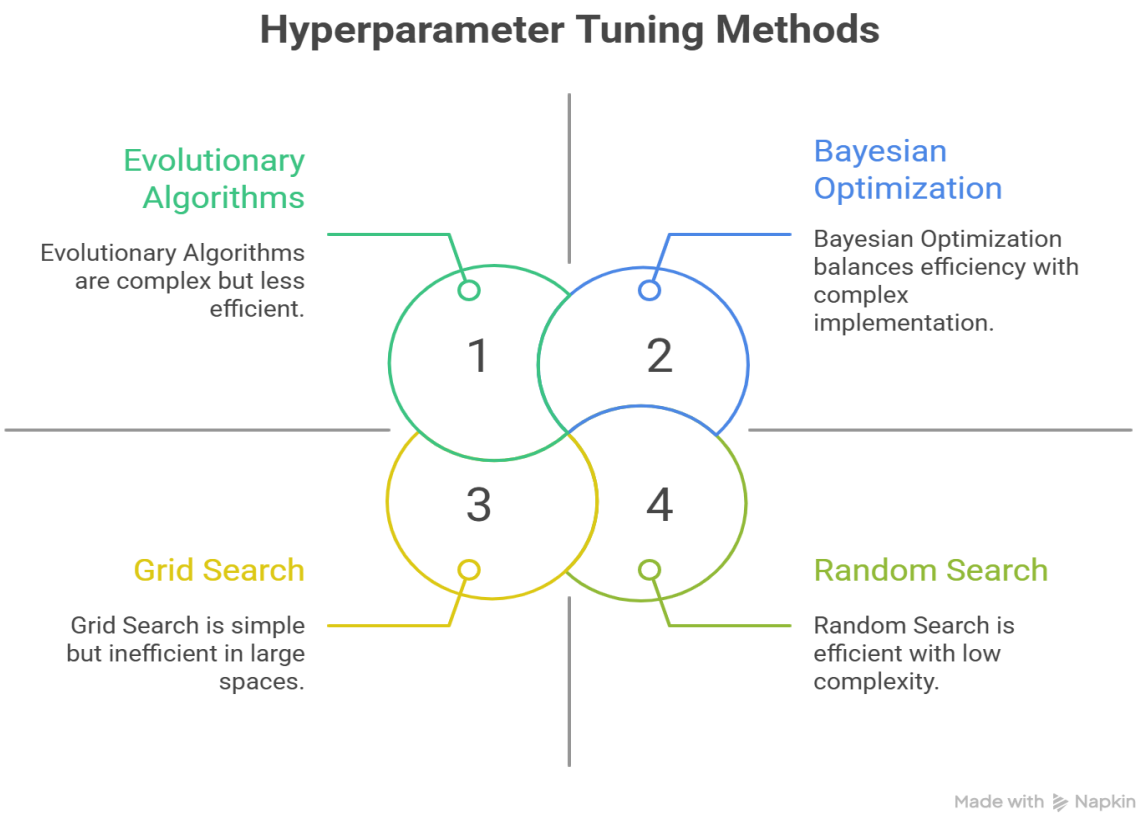
🔹 Regularization

- Dropout rate.
- Weight decay (L2 penalty).
-
- Early stopping.

🔹 Model Architecture

- Number of layers.
- Neurons per layer.
- Activation functions.

Methods of Hyperparameter Tuning



- Grid Search
  - Tests all combinations from predefined hyperparameter values.
  - **Pros:** Exhaustive, finds best within the grid.
  - **Cons:** Very expensive in large spaces.
  - **Example:**
    - Learning rate: [0.01, 0.001]
    - Batch size: [16, 32]
    - Total = 4 combinations.

🔹 Random Search

- Chooses random combinations of hyperparameters.
- **Pros:** More efficient, works better for high dimensions.
- **Cons:** May miss exact best configuration.
- **Practical fact:** Random search is often as good or better than grid search for deep learning.

🔹 Bayesian Optimization

- Builds a probabilistic model of performance (objective function).
- Selects hyperparameters by balancing exploration vs exploitation.
- **Pros:** Needs fewer trials.
- **Cons:** Complex implementation.
- **Example libraries:** HyperOpt, Optuna, Spearmint.

🔹 Evolutionary Algorithms

- Inspired by genetics: mutation, crossover, selection.
- Models "evolve" better hyperparameters over generations.
- **Pros:** Works in complex search spaces.
- **Cons:** Computationally expensive.

🔹 Automated Hyperparameter Tuning (AutoML)

- Uses frameworks to automate tuning at scale.
- **Examples:** Keras Tuner, Optuna, Ray Tune, Google Vizier.
- **Pros:** Saves time, production-ready.
- **Cons:** Requires more compute resources.

Practical Workflow for Hyperparameter Tuning

1. **Start Simple**
    - Use known defaults (Adam optimizer, lr=0.001, batch=32).
2. **Tune Learning Rate First**
    - Most sensitive hyperparameter.
    - Use a learning rate finder (train for a few steps with increasing lr).
3. **Tune Batch Size Next**
    - Try small (32), medium (64), large (128).
4. **Add Regularization**
    - Dropout, weight decay, early stopping.
5. **Use Random Search or Bayesian Optimization**
    - Grid search is inefficient in high dimensions.

6. **Use Early Stopping**
    - Stop bad trials early to save time.
7. **Automate with Tools**
    - Optuna, Keras Tuner, Ray Tune.

Rule of Thumb: 80% of impact comes from learning rate, batch size, and regularization.

Example: Hyperparameter Tuning with Optuna

```
import optuna
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader


# Define simple model
class SimpleNN(nn.Module):
    def __init__(self, hidden_size):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(28*28, hidden_size)
        self.fc2 = nn.Linear(hidden_size, 10)
```

```
    def forward(self, x):
        x = x.view(-1, 28*28)
        x = torch.relu(self.fc1(x))
        return torch.log_softmax(self.fc2(x), dim=1)

# Objective function for tuning
def objective(trial):
    hidden_size = trial.suggest_int('hidden_size', 32, 256)
    lr = trial.suggest_loguniform('lr', 1e-4, 1e-1)
    batch_size = trial.suggest_categorical('batch_size', [32, 64, 128])

    transform = transforms.ToTensor()
    train_data = datasets.MNIST('./data', train=True, download=True, transform=transform)
    train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True)

    model = SimpleNN(hidden_size)
    optimizer = optim.Adam(model.parameters(), lr=lr)
    criterion = nn.NLLLoss()

    # Train for 1 epoch (short for tuning)
    for images, labels in train_loader:
        optimizer.zero_grad()
        output = model(images)
```

```
        loss = criterion(output, labels)
        loss.backward()
        optimizer.step()
        break  # just one batch for quick evaluation
    return loss.item()
# Run tuning
study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=20)
print("Best hyperparameters:", study.best_params)
```

Real-World Applications

- Computer Vision (CNNs) → tuning filter sizes, stride, learning rate.
- NLP (Transformers) → hidden size, number of layers, learning rate warmup.
- Healthcare → maximizing recall for disease prediction.
- Finance → minimizing false negatives in fraud detection.